Media Ratings Platform (MRP) – Specification

Create a standalone application in **Java** or **C#** that implements a **RESTful HTTP server** acting as an API backend for possible clients (Web, Mobile, Console, etc.).

The frontend is **not part of this project**.

The goal of this project is to design and implement a moderately complex software system using concepts from SWEN1, including:

- Object-Oriented Programming
- Layered Architecture
- REST/HTTP
- PostgreSQL persistence
- SOLID principles
- Repository pattern
- Testing and mocking

Features

This HTTP/REST-based server provides a platform where users can manage media content (movies, series, and games), rate them, and receive recommendations.

Users

- register and log in using unique credentials (username, password)
- receive an authorization token after login
- can view and edit a profile with personal statistics
- can create, update, and delete media entries
- can rate media entries from 1–5 stars
- can optionally write comments for ratings
- can edit or delete their own ratings
- can like ratings of other users
- may only like a rating once
- can mark media entries as favorites
- can remove media entries from favorites
- can view personal rating history, favorites, and statistics
- receive recommendations based on previously highly rated media and content similarity

Media entries

- represent either a movie, series, or game
- consist of title, description, media type, release year, genre(s), and age restriction
- may only be edited or deleted by their creator
- store ratings, average score, and favorite count
- can be marked as favorite by other users

Ratings

- belong to exactly one user and one media entry
- contain star value (1–5), optional comment, and timestamp
- may be edited or deleted only by their creator
- may receive likes from other users
- are initially hidden until confirmed by the creator

Search, Filtering & Sorting

- partial title matching
- filtering by genre, media type, release year, age restriction, and minimum rating
- sorting by title, release year, and average score
- implementation in SQL, application logic, or both

Recommendations

- must consider previously highly rated media
- should consider genre similarity, media type similarity, and age restriction similarity
- recommendation logic must be documented

Statistics & Leaderboard

- user statistics such as total ratings, average rating, favorite genre
- public leaderboard sorted by number of ratings

Authorization Rules

- protected endpoints require authentication
- users may only edit/delete their own ratings
- users may only edit/delete their own media entries
- one like per user per rating

Technical Requirements

- build a RESTful HTTP server
- allowed: **HttpListener** and lightweight helper libraries
- **not allowed**: ASP.NET, Spring, JSP/JSF, other full-stack frameworks

API Requirements

- use JSON for requests and responses
- use proper HTTP methods (GET, POST, PUT, DELETE)
- use meaningful HTTP status codes
- use REST-style routes

Token-Based Authorization

- token-based authentication is required
- tokens must be sent using Authorization: Bearer <token>
- The server must validate token existence, validity, and authorization

Persistence

- all persistent data must be stored in PostgreSQL
- PostgreSQL must run inside Docker

Testing Requirements

- create at least 20 automated unit tests
- tests should cover authorization, filtering, recommendations, and business rules
- mocking frameworks may be used

Hand-Ins

The submission is done in two steps:

1. **Intermediate-Submission** (class 13): Includes HTTP server, user registration/login, token handling, CRUD, PostgreSQL persistence
2. **Final Submission** (class 22): adds full business logic, rating system, favorites, filtering, recommendations, leaderboard

Hand in the latest version of your source code as a zip in Moodle that includes:

- Source code
- README.txt or README.md with link to GitHub repository
- Postman collection or curl-script demonstrating all relevant endpoints
- protocol. with a detailed development report

Protocol Document

- Description of technical steps and architecture decisions
- Explanation of unit test coverage and why specific logic was tested
- Notes on problems encountered and how they were solved
- Estimated time tracking for each major part of the project
- consider that the git-history is part of the documentation (no need to copy it into the protocol)

See the corresponding Checklist Excel sheets for the MUST-HAVES, Grading-Items and Grading-Points. Late submissions are not accepted! Missing hand-ins or missing MUST-HAVEs will automatically grade the submission with 0 points!

Final Presentation

For the final presentation in class 22-25, be prepared with your

- working solution already started on your machine.
- set up your environment so you can start the postman or curl tests directly.
- open your design (see protocol) to show your architecture / approach.

Optional Features

With optional features implemented you can compensate for possible errors in the implementation above. Nevertheless, it is not possible to exceed the maximum number of points (= 100%).